

T.C.
MARMARA ÜNİVERSİTESİ
TEKNOLOJİ FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ



ARAÇ KİRALAMA SİSTEMİ

Bilgisayar Programlama II Laboratuvarı Projesi

SONUÇ RAPORU

Araştırmacıların Adı Soyadı
Birimi/Bölümü
Öğrenci numarası
Ferhat Kara Bilgisayar Mühendisliği
170425508
Raul Kazımlı Bilgisayar Mühendisliği
170424950

Dersi Veren:
Doç.Dr. Ayşe Berna Altınel Girgin
Arş. Gör. Şeyda Karcı

İÇİNDEKİLER

ÖZET	
GİRİŞ	
KULLANILAN TEKNOLOJİLER	
SİSTEM TASARIMI VE MİMARİSİ	
MVC Tasarım Deseni	
Veri Yapıları ve Dosya Sistemi	
Kullanıcı Modeli ve Yetkilendirme	
GERÇEKLEŞTİRİM VE FONKSİYONLAR	
ARAMA VE FİLTRELEME ALGORİTMASI	
TESTLER VE SENARYOLAR	
SONUÇ VE GELECEK PLANLARI	

1. ÖZET

Bu proje kapsamında, Python programlama dili kullanılarak masaüstü ortamda çalışan, MVC (Model–View–Controller) mimarisine uygun bir araç kiralama ve araç paylaşım sistemi geliştirilmiştir. Geliştirilen uygulama, kullanıcıların yalnızca araç kiraladığı klasik sistemlerin ötesine geçerek, her kullanıcının sisteme kendi aracını ekleyebildiği ve diğer kullanıcıların araçlarını kiralayabildiği paylaşım temelli bir yapı sunmaktadır. Bu yönüyle sistem, kullanıcılar arası etkileşimi artıran ve kaynakların daha verimli kullanılmasını hedefleyen modern araç paylaşım platformlarına benzer bir yaklaşım benimsemektedir.

Uygulamanın arayüz tasarımı Python'un Tkinter kütüphanesi kullanılarak gerçekleştirilmiş, kullanıcı dostu ve sade bir masaüstü deneyimi amaçlanmıştır. Sistem mimarisi MVC tasarım deseni temel alınarak oluşturulmuş; veri işlemleri Model katmanında, kullanıcı etkileşimleri View katmanında ve iş mantığı ile yönlendirme süreçleri Controller katmanında ayrı ayrı ele alınmıştır. Bu yapı sayesinde kodun okunabilirliği, sürdürülebilirliği ve geliştirilebilirliği artırılmıştır.

Veri yönetimi amacıyla geleneksel veritabanları yerine CSV (Comma Separated Values) tabanlı bir dosya sistemi tercih edilmiştir. Kullanıcı bilgileri, araç kayıtları, kiralama logları, sepet verileri ve işlem geçmişleri ayrı CSV dosyalarında tutulmuştur. Bu yaklaşım, küçük ve orta ölçekli masaüstü uygulamalar için yeterli performans sağlarken, veri işlemlerinin şeffaf ve kolay yönetilebilir olmasına olanak tanımıştır.

Sistem; kullanıcı kayıt ve giriş işlemleri, şifre değiştirme ve şifre kurtarma (SMTP tabanlı e-posta gönderimi), araç ekleme ve silme, tarih bazlı araç kiralama, iade işlemleri, sepet yönetimi ve günlük işlem raporlarının görüntülenmesi gibi birçok temel ve gelişmiş fonksiyonu içermektedir. Ayrıca tarih çakışmalarını önlemek amacıyla kiralama uygunluk kontrolleri gerçekleştirilmiş, hatalı girişlerin önüne geçmek için plaka, tarih ve ödeme bilgilerine yönelik doğrulama (validation) mekanizmaları uygulanmıştır.

Sonuç olarak bu proje, yazılım mühendisliği prensiplerine uygun, modüler ve genişletilebilir bir masaüstü araç kiralama sistemi ortaya koymaktadır. Geliştirilen yapı, ilerleyen aşamalarda gerçek bir veritabanı entegrasyonu, web veya mobil platformlara taşınma ve gelişmiş raporlama özellikleri eklenerek daha kapsamlı bir sisteme dönüştürülebilecek altyapıyı sağlamaktadır.

Anahtar Kelimeler: Python, Tkinter, MVC, CSV, Araç Kiralama, Araç Paylaşımı, SMTP, Masaüstü Uygulama.

2. GİRİŞ

Günümüzde araç kiralama sistemleri, artan kullanıcı sayısı ve veri hacmi nedeniyle güvenilir veri yönetimi, işlem doğruluğu ve kullanıcı deneyimi açısından yüksek standartlar gerektirmektedir. Özellikle dijitalleşmenin yaygınlaşmasıyla birlikte, kullanıcıların hızlı, güvenli ve hatasız işlem yapabilmesini sağlayan yazılım çözümleri büyük önem kazanmıştır. Bu bağlamda geliştirilen yazılım sistemlerinin yalnızca işlevsel olması değil, aynı zamanda sürdürülebilir, ölçeklenebilir ve geliştirilmeye açık bir mimariye sahip olması beklenmektedir.

Bu proje, söz konusu ihtiyaçları karşılamak amacıyla nesne yönelimli programlama (Object-Oriented Programming – OOP) prensipleri temel alınarak tasarlanmıştır. OOP yaklaşımı sayesinde sistem; modüler, okunabilir ve bakımı kolay bir yapıya kavuşturulmuş, ilerleyen

süreçlerde yeni özelliklerin eklenebilmesine uygun hâle getirilmiştir. Sınıflar ve fonksiyonlar aracılığıyla gerçek hayat senaryoları yazılım dünyasına modellenmiş; kullanıcı, araç ve kiralama kavramları ayrı nesneler olarak ele alınmıştır.

Geliştirilen uygulamanın en dikkat çekici özelliklerinden biri, klasik araç kiralama sistemlerinden farklı olarak merkezi bir yönetici (admin) yapısına bağlı kalmamasıdır. Bunun yerine, kullanıcıların hem araç sahibi hem de kiracı rolünü üstlenebildiği hibrit bir sistem mimarisi benimsenmiştir. Bu yaklaşım, araç paylaşım mantığına dayalı modern platformların temel çalışma prensiplerini yansıtmaktadır. Kullanıcılar sisteme giriş yaptıktan sonra kendi araçlarını sisteme ekleyebilmekte, diğer kullanıcıların araçlarını görüntüleyerek kiralayabilmekte veya araçları sepetlerine ekleyerek daha sonra işlem gerçekleştirebilmektedir.

Uygulama içerisinde kullanıcı etkileşimlerini kolaylaştırmak amacıyla sade ve anlaşılır bir arayüz tasarlanmış, işlemler mümkün olduğunca minimum adımda tamamlanacak şekilde kurgulanmıştır. Kullanıcı doğrulama, tarih bazlı kiralama, sepet yönetimi ve iade işlemleri gibi kritik süreçlerde hata payını azaltmak için çeşitli kontrol ve doğrulama mekanizmaları uygulanmıştır. Bu sayede hatalı tarih seçimi, çakışan kiralama talepleri ve geçersiz veri girişlerinin önüne geçilmesi hedeflenmiştir.

Projenin teknik altyapısında veri bütünlüğünü ve işlem takibini sağlamak adına kapsamlı bir loglama sistemi kurulmuştur. Kullanıcıların gerçekleştirdiği araç ekleme, silme, kiralama ve iade işlemleri hem anlık kayıtlar (logs) hem de geçmiş işlemler (history) üzerinden detaylı şekilde kayıt altına alınmaktadır. Bu yapı, sistemin denetlenebilirliğini artırmakta ve olası hataların geriye dönük olarak analiz edilebilmesine imkân tanımaktadır.

Sonuç olarak bu proje, yazılım mühendisliği ilkeleri doğrultusunda tasarlanmış, kullanıcı odaklı ve paylaşım temelli bir araç kiralama uygulaması sunmaktadır. Giriş bölümünde ele alınan bu temel yaklaşım, ilerleyen bölümlerde sistem mimarisi, kullanılan teknolojiler ve uygulama detaylarıyla daha kapsamlı şekilde açıklanacaktır.

3. KULLANILAN TEKNOLOJİLER

Bu projede, masaüstü ortamda çalışan, modüler ve sürdürülebilir bir araç kiralama uygulaması geliştirmek amacıyla çeşitli yazılım teknolojilerinden ve Python'un standart kütüphanelerinden faydalanılmıştır. Seçilen teknolojiler, projenin kapsamı, hedeflenen kullanıcı kitlesi ve geliştirme sürecinin verimliliği göz önünde bulundurularak belirlenmiştir.

3.1. Python 3.x

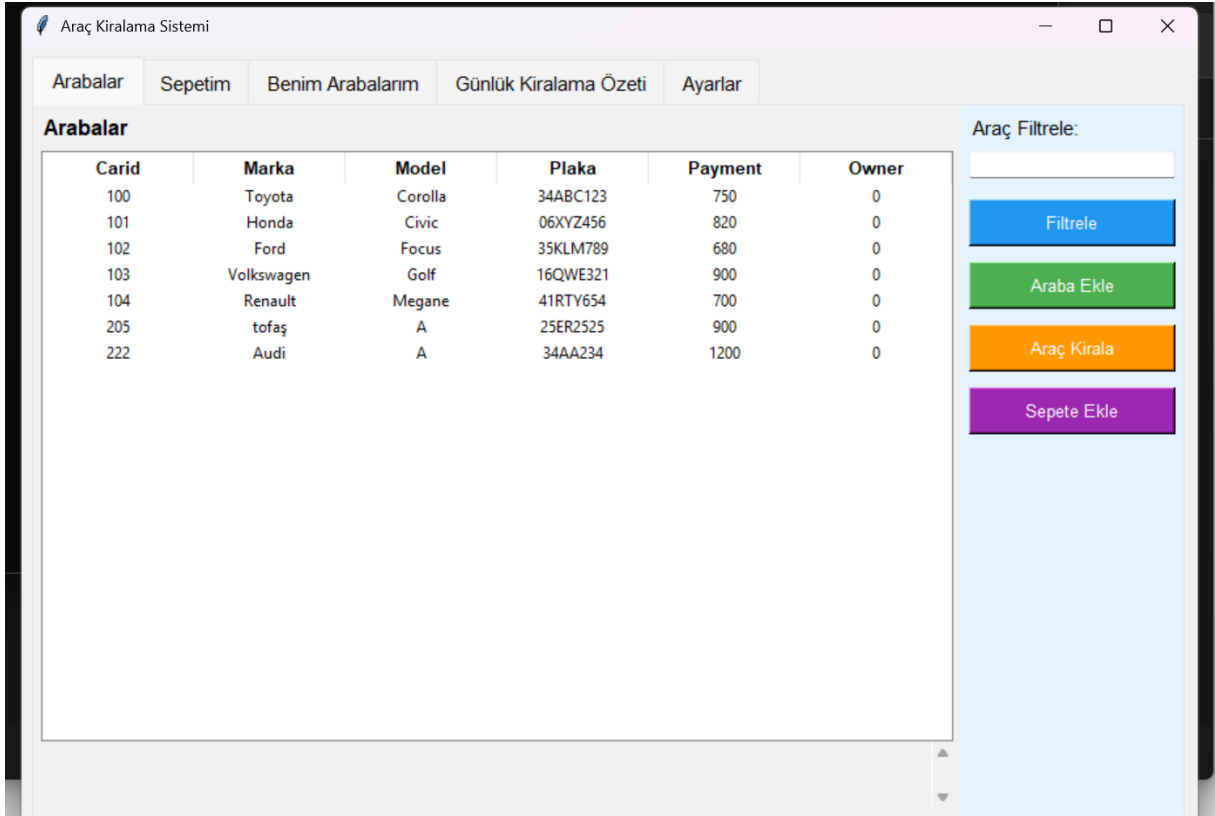
Python 3.x, projenin temel geliştirme dili olarak tercih edilmiştir. Python'un sade ve okunabilir söz dizimi, hızlı prototipleme imkânı sunması ve geniş kütüphane desteği, projede etkin bir şekilde kullanılmasını sağlamıştır. Nesne yönelimli programlama (OOP) yapısına uygun olması sayesinde kullanıcı, araç ve kiralama gibi temel kavramlar sınıflar aracılığıyla modellenmiştir.

Projede Python dili; iş mantığının oluşturulması, veri işleme süreçleri, tarih kontrolleri, dosya okuma-yazma işlemleri ve e-posta gönderimi gibi tüm temel fonksiyonların gerçekleştirilmesinde kullanılmıştır. Ayrıca Python'un platform bağımsız yapısı, uygulamanın farklı işletim sistemlerinde sorunsuz çalışabilmesine olanak tanımaktadır.

3.2. Tkinter

Tkinter, Python'un standart grafiksel kullanıcı arayüzü (GUI) kütüphanesi olup, bu projede View (Görünüm) katmanının geliştirilmesinde kullanılmıştır. Tkinter sayesinde kullanıcıların sisteme giriş yapabileceği, araçları listeleyebileceği, sepet işlemlerini gerçekleştirebileceği ve kiralama/iade işlemlerini yönetebileceği etkileşimli bir masaüstü arayüz tasarlanmıştır.

Kütüphanenin Python ile entegre çalışması, harici bağımlılıklara ihtiyaç duyulmaması ve öğrenme eğrisinin düşük olması, Tkinter'ın tercih edilmesinde etkili olmuştur. Arayüz tasarımında sadelik ve kullanılabilirlik ön planda tutulmuş, kullanıcıların işlemleri hızlı ve anlaşılır şekilde gerçekleştirmesi hedeflenmiştir.



3.3. CSV (Comma Separated Values) Dosya Sistemi

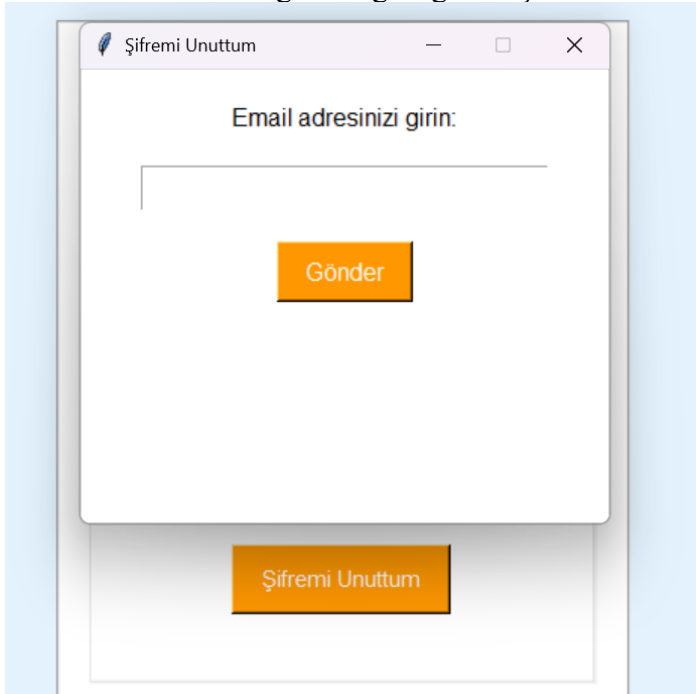
Projede veri tabanı olarak geleneksel ilişkisel veritabanları yerine CSV tabanlı bir dosya sistemi kullanılmıştır. Kullanıcı bilgileri, araç kayıtları, kiralama logları, sepet verileri ve işlem geçmişleri ayrı CSV dosyalarında saklanmaktadır. Bu yapı, küçük ve orta ölçekli masaüstü uygulamalar için yeterli performans sunmakta ve veri işlemlerinin basit ve şeffaf bir şekilde yönetilmesini sağlamaktadır.

Python'un pandas kütüphanesi kullanılarak CSV dosyaları üzerinde okuma, yazma, filtreleme ve güncelleme işlemleri gerçekleştirilmiştir. Bu sayede veriler üzerinde hızlı işlem yapılabilmiş ve veri bütünlüğü korunmuştur. CSV dosya yapısı, aynı zamanda sistemin taşınabilirliğini artırmış ve harici bir veritabanı kurulumuna ihtiyaç duyulmamasını sağlamıştır.

3.4. SMTP (smtplib)

SMTP (Simple Mail Transfer Protocol), projede “Şifremi Unuttum” fonksiyonunun gerçekleştirilmesi amacıyla kullanılmıştır. Python’un smtplib modülü aracılığıyla, kullanıcının sistemde kayıtlı e-posta adresine otomatik olarak şifre gönderimi yapılmaktadır. Bu özellik, kullanıcı deneyimini iyileştiren ve sistemin güvenilirliğini artıran önemli bir fonksiyon olarak projeye entegre edilmiştir.

E-posta gönderim sürecinde, Gmail SMTP sunucuları kullanılmış; bağlantı güvenliği için TLS (Transport Layer Security) protokolü tercih edilmiştir. Böylece kullanıcı bilgilerinin iletimi sırasında veri güvenliği sağlanmıştır.



3.5. Datetime Modülü

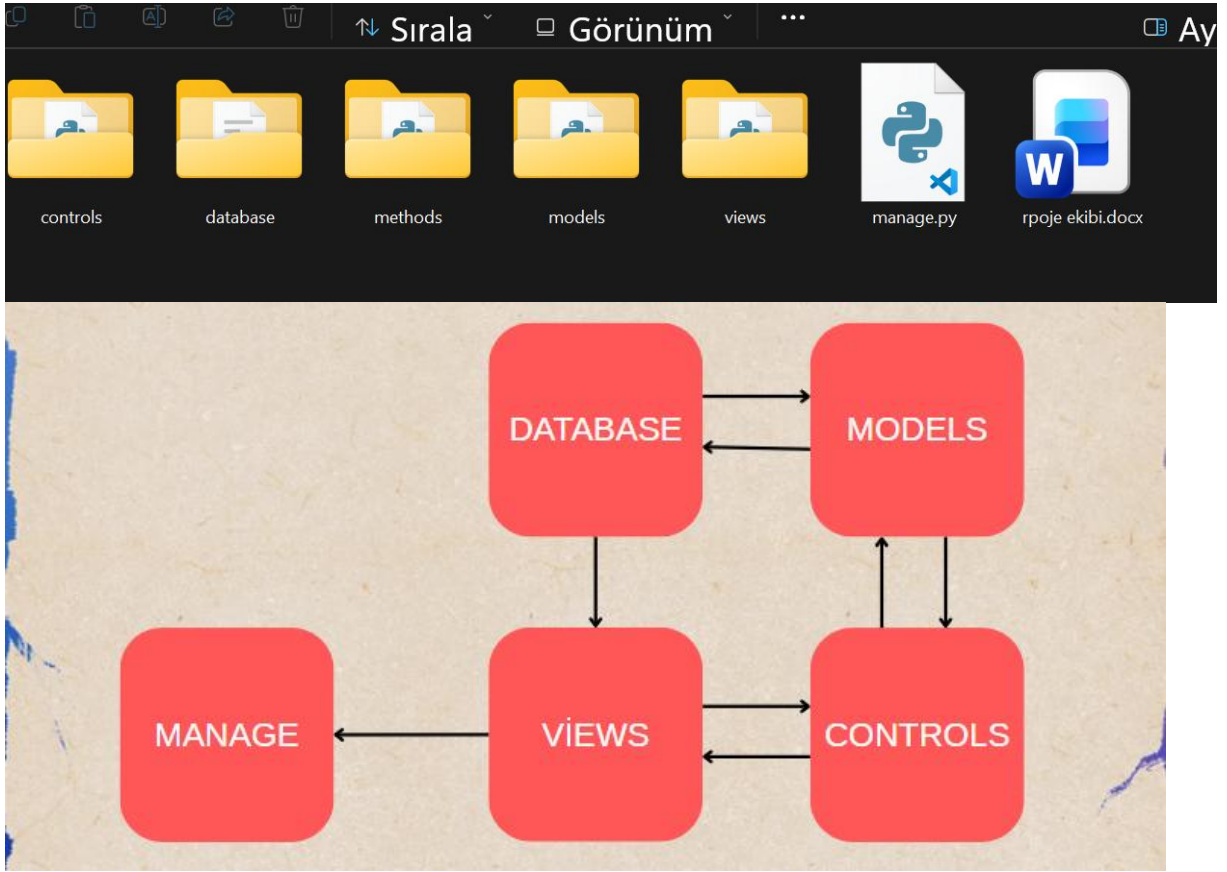
Python’un datetime modülü, sistemdeki tarih ve zaman tabanlı işlemlerin yönetilmesinde kullanılmıştır. Araç kiralama başlangıç ve bitiş tarihleri, iade tarihleri ve günlük işlem raporları bu modül sayesinde kontrol altına alınmıştır. Tarih karşılaştırmaları yapılarak geçmiş tarihli kiralama veya hatalı iade işlemlerinin önüne geçilmiştir.

Ayrıca sistemde gerçekleştirilen işlemlerin tarih ve saat bilgileri log kayıtlarına eklenerek, işlem geçmişinin detaylı bir şekilde izlenmesi sağlanmıştır. Bu yapı, hem kullanıcı bazlı raporlama hem de sistem denetimi açısından önemli katkılar sunmaktadır.

4. SİSTEM TASARIMI VE MİMARİSİ

Bu projede, yazılımın okunabilirliğini, bakımını ve ileride geliştirilebilirliğini artırmak amacıyla **MVC (Model–View–Controller)** tasarım deseni benimsenmiştir. MVC mimarisi sayesinde uygulama; veri yönetimi, kullanıcı arayüzü ve iş mantığı olmak üzere üç temel katmana ayrılmıştır. Bu ayırım, kod tekrarını azaltmış, hata ayıklamayı kolaylaştırmış ve sistemin modüler bir yapıya kavuşmasını sağlamıştır.

Uygulamanın genel mimarisi, kullanıcıdan gelen etkileşimlerin önce **View** katmanında alınması, ardından **Controller** katmanında işlenmesi ve gerekli durumlarda **Model** katmanı üzerinden veri güncellenmesi prensibi üzerine kurulmuştur.



4.1. MVC Tasarım Deseni

MVC tasarım deseni, uygulamanın farklı sorumluluklarını ayrı katmanlara bölerek daha düzenli ve sürdürülebilir bir yazılım geliştirme süreci sunar. Bu projede MVC katmanları aşağıdaki şekilde yapılandırılmıştır:

Model (Models Katmanı)

Model katmanı, sistemde kullanılan temel veri yapılarının ve iş kurallarının tanımlandığı bölümdür. Bu katmanda özellikle **User** ve **Car** sınıfları yer almaktadır. Kullanıcı bilgileri, araç verileri, kiralama işlemleri ve iade süreçleri bu sınıflar aracılığıyla yönetilmektedir.

Model katmanı, CSV dosyaları ile doğrudan etkileşim hâlinindedir. Verilerin okunması, güncellenmesi ve kaydedilmesi işlemleri bu katmanda gerçekleştirilerek veri bütünlüğü sağlanmıştır. Böylece View ve Controller katmanlarının veri yapılarının detaylarıyla doğrudan ilgilenmesine gerek kalmamıştır.

View (Görünüm Katmanı)

View katmanı, kullanıcı ile sistem arasındaki etkileşimin gerçekleştiği katmandır. Bu projede View katmanı **Tkinter** kütüphanesi kullanılarak geliştirilmiştir. Giriş ekranları, araç listeleme sayfaları, sepet ekranı, kiralama ve iade formları bu katmanda yer almaktadır.

View katmanı yalnızca kullanıcıdan veri almak ve kullanıcıya çıktı göstermekle sorumludur. İş mantığına veya veri işleme detaylarına doğrudan müdahale etmez. Bu yaklaşım, arayüz tasarımında yapılacak değişikliklerin sistemin diğer bölümlerini etkilememesini sağlamaktadır.

Controller (Kontrolcü Katmanı)

Controller katmanı, View'dan gelen kullanıcı eylemlerini işleyen ve gerekli durumlarda Model katmanı ile iletişim kuran yapıdır. Buton tıklamaları, arama işlemleri, araç kiralama ve iade talepleri bu katmanda kontrol edilmektedir.

Controller katmanında ayrıca giriş doğrulamaları (validation), tarih kontrolleri ve hata yönetimi gibi kritik işlemler gerçekleştirilmiştir. Böylece sistemde hatalı veri girişlerinin önüne geçilmiş ve kullanıcıya anlamlı geri bildirimler sunulmuştur.

4.2. Veri Yapıları ve Dosya Sistemi

Projede veri tabanı olarak ilişkisel bir yapı simüle edilmiş ve kalıcı veri saklama amacıyla **CSV tabanlı dosya sistemi** kullanılmıştır. Toplamda beş adet temel CSV dosyası bulunmaktadır ve her biri sistemin farklı bir fonksiyonunu temsil etmektedir:

1. **auth.csv**
Kullanıcıların kimlik bilgilerini saklar.
İçerik: Kullanıcı ID, e-posta adresi, kullanıcı adı, şifre ve oturum durumu (islogin).
2. **cars.csv**
Sisteme kayıtlı tüm araçların bilgilerini içerir.
İçerik: Araç ID, marka, model, plaka, günlük ücret ve araç sahibi bilgisi.
3. **sepet.csv**
Kullanıcıların kiralama öncesinde seçtikleri araçları geçici olarak tuttuğu alandır.
Her kullanıcı yalnızca kendi ID'si ile eşleşen kayıtları görüntüleyebilir.
4. **logs.csv**
Aktif kiralama işlemlerinin tutulduğu dosyadır.
Araçların hangi tarihler arasında kiralandığı ve iade durumları burada takip edilir.
5. **history.csv**
Sistem üzerinde gerçekleştirilen tüm işlemlerin kayıt altına alındığı geçmiş (audit log) dosyasıdır.
Araç ekleme, silme, kiralama ve iade işlemleri tarih ve saat bilgisiyle birlikte saklanır.

Bu yapı sayesinde sistemde yapılan tüm işlemler izlenebilir hâle getirilmiş ve veri bütünlüğü korunmuştur.

```
C:\Users\FERHAT KARA\OneDrive\Masaüstü\rent\database\history.csv (preview)
1 id,did,value,carid,date,time
2 838582,araç eklendi,200,333,22-12-25,00:30
3 838582,araç eklendi,1111 TL,1111,22-12-25,00:35
4 838582,araç eklendi,2222 TL,2222,22-12-25,00:36
5 838582,kiralama tamamlandı,12-12-30-21-12-30,100000
6 838582,kiralama tamamlandı,24-12-25-28-12-25,100000
7 838582,iade tarihi güncellendi,26-12-25,102,22-12-25
8 775950,kiralama tamamlandı,01-01-26-10-01-26,110000
9 775950,araç eklendi,12 TL,10,23-12-25,21:13
10 775950,kiralama tamamlandı,10-10-30-20-10-30,205000
11

logs.csv > data
1 carid,datestart,dateend,iade,rentedby
2 101,11-11-25,22-12-25,-,100000
3 103,03-03-25,04-03-25,-,100000
4 103,10-03-25,15-03-25,-,100000
5 205,10-10-25,12-12-25,20-10-25,0
6 222,10-10-25,12-12-25,20-10-25,0
7 222,15-12-25,17-12-25,-,0
8 222,01-12-25,05-12-25,-,0
9 103,12-12-30,21-12-30,-,838582
10 102,24-12-25,28-12-25,26-12-25,838582
11 100,20-12-25,25-12-25,-,100000
12 1111,01-01-26,10-01-26,-,775950
13 205,10-10-30,20-10-30,-,775950
14

auth.csv > data
1 id,email,name,password,islogin
2 100000,fkara74@icloud.com,ferhat-kara,q,0
3 706389,ferhaatkaaraa@gmail.com,ferhat,yenisifre,1
4 775950,7fkara@gmail.com,fero,123,1
5 650699,kazimliraul404@gmail.com,ahmet,123ahmet,0
6

cars.csv > data
1 carid,marka,model,plaka,payment,owner
2 100,Toyota,Corolla,34ABC123,750,0
3 101,Honda,Civic,06XYZ456,820,0
4 102,Ford,Focus,35KLM789,680,0
5 103,Volkswagen,Golf,16QWE321,900,0
6 104,Renault,Megane,41RTY654,700,0
7 205,tofaş,A,25ER2525,900,0
8 222,Audi,A,34AA234,1200,0
9
```

4.3. Kullanıcı Modeli ve Yetkilendirme

Sistemde geleneksel bir **admin-kullanıcı** ayrımı bulunmamaktadır. Bunun yerine tüm kullanıcılar eşit yetkilere sahiptir ve sistemin aktif bir parçası olarak tasarlanmıştır. Bu yaklaşım, uygulamanın paylaşım temelli yapısını desteklemektedir.

Kullanıcı yetkilendirme kuralları aşağıdaki şekilde belirlenmiştir:

- Kullanıcılar sisteme kendi araçlarını ekleyebilir.
- Kullanıcılar yalnızca **kendilerine ait** araçları silebilir.
- Kullanıcılar hem kendi araçlarını hem de diğer kullanıcıların araçlarını kiralayabilir.
- Her kullanıcı sadece kendi sepetini ve kendi işlem geçmişini görüntüleyebilir.

Bu yetkilendirme yapısı, hem güvenliği sağlamakta hem de sistemin adil bir şekilde kullanılmasına olanak tanımaktadır.

5. GERÇEKLEŞTİRİM VE FONKSİYONLAR

Bu bölümde, geliştirilen araç kiralama uygulamasının temel fonksiyonları, bu fonksiyonların çalışma mantığı ve sistem içerisindeki rolleri detaylı olarak açıklanmaktadır. Projede yer alan tüm işlevler, MVC mimarisi doğrultusunda Model ve Controller katmanlarında gerçekleştirilmiş; View katmanı yalnızca kullanıcıdan veri almak ve sonuçları göstermekle sınırlandırılmıştır.

5.1. Kullanıcı Yönetimi (Kayıt, Giriş, Çıkış ve Şifre İşlemleri)

Sistemde kullanıcı işlemleri User sınıfı üzerinden yürütülmektedir. Yeni bir kullanıcı sisteme kayıt olmak istediğinde `signin()` fonksiyonu çağrılır. Bu fonksiyon, öncelikle `auth.csv` dosyasını kontrol ederek e-posta adresi ve kullanıcı adının daha önce kayıtlı olup olmadığını denetler. Çakışma yoksa kullanıcıya rastgele ve benzersiz bir ID atanır ve bilgiler dosyaya kaydedilir.

Kullanıcı girişi işlemi `login()` fonksiyonu ile gerçekleştirilir. Bu fonksiyon, kullanıcı adı veya e-posta adresi ile şifre eşleşmesini kontrol eder. Başarılı giriş durumunda kullanıcının `islogin` alanı güncellenir ve oturum bilgileri sisteme aktarılır. Çıkış işlemi ise `logout()` fonksiyonu ile sağlanır ve kullanıcının aktif oturumu sonlandırılır.

Şifre değiştirme işlemleri `changepass()` fonksiyonu üzerinden yürütülür. Kullanıcıdan eski şifre doğrulaması alınarak yeni şifre güncellenir. Bu mekanizma, yetkisiz şifre değişikliklerinin önüne geçmektedir.

5.2. Şifre Kurtarma Mekanizması (sendmail)

Kullanıcı şifresini unuttuğunda, sistem SMTP tabanlı e-posta gönderimi ile şifre kurtarma işlemi sunmaktadır. `sendmail()` fonksiyonu, kullanıcının girdiği e-posta adresini `auth.csv` dosyasında sorgular. Eğer e-posta adresi sistemde kayıtlıysa, ilgili kullanıcıya mevcut şifresi otomatik olarak e-posta yoluyla gönderilir.

Bu işlem sırasında Python'un `smtplib` modülü kullanılmıştır. Gmail SMTP sunucuları üzerinden TLS güvenliği ile bağlantı kurulmuş, e-posta içeriği UTF-8 formatında gönderilmiştir. E-posta adresi sistemde bulunmadığında ise kullanıcıya hata mesajı döndürülmektedir.

5.3. Araç Ekleme ve Silme İşlemleri

Araç yönetimi işlemleri Car sınıfı üzerinden gerçekleştirilmiştir. Yeni bir araç ekleme işlemi sırasında araç ID, plaka formatı ve ücret bilgileri çeşitli doğrulama fonksiyonları ile kontrol edilmektedir. Geçerli veriler girildiğinde araç cars.csv dosyasına eklenir.

Araç ekleme ve silme işlemleri yalnızca aracın sahibi tarafından yapılabilir. Her araç ekleme veya silme işlemi, history.csv dosyasına tarih ve saat bilgisiyle birlikte kaydedilerek sistemde izlenebilirlik sağlanmıştır.

Carid	Marka	Model	Plaka	Payment	Owner
100	Toyota	Corolla	34ABC123	750	0
101	Honda				
102	Ford				
103	Volkswagen				
104	Renault				
205	tofaş				
222	Audi				

5.4. Kiralama ve Sepet Mantığı (_rent, add_sepet, rm_sepet)

Kiralama sistemi, projenin en kritik ve karmaşık bileşenlerinden biridir. Kullanıcı, bir aracı iki farklı şekilde değerlendirebilir: doğrudan kiralama veya sepete ekleme.

add_sepet() fonksiyonu, kullanıcının beğendiği aracı geçici olarak sepet.csv dosyasına eklemesini sağlar. Sepet yapısı, kullanıcıya karar verme sürecinde esneklik sunar. Sepetteki araçlar tekil olarak yönetilir; kullanıcı isterse seçilen aracı kiralayabilir ya da sepetten çıkarabilir.

Doğrudan kiralama işlemi _rent() fonksiyonu ile gerçekleştirilir. Bu fonksiyon, seçilen aracın belirtilen tarih aralığında başka bir kullanıcı tarafından kiralanıp kiralanmadığını kontrol eder. Bu kontrol, daha önce oluşturulmuş canirent() fonksiyonu ile sağlanmaktadır. Tarih çakışması yoksa kiralama işlemi başarılı şekilde tamamlanır ve işlem logs.csv ile history.csv dosyalarına kaydedilir.

Bu mekanizma sayesinde aynı aracın aynı tarihlerde birden fazla kişi tarafından kiralanması engellenmiştir.

Arabalar

Sepetim

Benim Arabalarım

Günlük Kiralama Özeti

Ayarlar

Sepetim

Carid	Marka	Model	Plaka	Payment	Owner
102	Ford	Focus	35KLM789	680	0
103	Volkswagen				0

Araç Kirala

Araç ID: 102

Başlangıç tarih:

Bitiş tarih:

Kirala

Araç Filtrele:

Filtrele

Sepeti Boşalt

Sepetten Çıkar

Araç Kirala

5.5. İade İşlemleri ve Esnek Tarih Yönetimi (_iade)

İade işlemleri _iade() fonksiyonu ile gerçekleştirilir. Kullanıcı, kiraladığı aracı taahhüt ettiği tarihten daha erken bir tarihte iade edebilir. Sistem, bu esnekliği desteklemek üzere tasarlanmıştır.

İade sırasında girilen tarih, kiralama başlangıç ve bitiş tarihleriyle karşılaştırılır. Eğer iade tarihi geçerli bir aralıkta ise, logs.csv dosyasında ilgili kaydın iade alanı güncellenir. Aynı zamanda işlem, history.csv dosyasına kaydedilerek geçmişte izlenebilir hâle getirilir.

Bu yapı, kullanıcıya esneklik sağlarken sistemdeki veri tutarlılığını da korumaktadır.

5.6. Arama, Filtreleme ve Raporlama Fonksiyonları

Sistem içerisinde kullanıcılar araçlar üzerinde arama ve filtreleme işlemleri gerçekleştirebilmektedir. _filter() fonksiyonu, CSV dosyaları üzerinde anahtar kelime bazlı arama yapılmasına olanak tanımaktadır. Bu sayede marka, model veya plaka bilgilerine göre hızlı sonuçlar elde edilebilmektedir.

Ayrıca günlük işlem raporları daily_rent() fonksiyonu aracılığıyla oluşturulmakta, kullanıcının aynı gün içerisinde gerçekleştirdiği işlemler listelenebilmektedir.

6. ARAMA VE FİLTRELEME ALGORİTMASI

Geliştirilen araç kiralama sisteminde, kullanıcı deneyimini artırmak amacıyla esnek ve kapsamlı bir arama-filtreleme algoritması uygulanmıştır. Bu algoritma, kullanıcının tek bir arama alanı üzerinden marka, model, plaka ve diğer metinsel alanlarda eş zamanlı arama yapabilmesini sağlamaktadır. Böylece kullanıcıların aradıkları araca hızlı ve pratik bir şekilde ulaşmaları hedeflenmiştir.

Arama mekanizması, kullanıcı tarafından arama çubuğuna girilen metnin (string) cars.csv dosyasında yer alan tüm sütunlarda kontrol edilmesi prensibine dayanmaktadır. Kullanıcının

girdiği ifade, belirli bir alanla sınırlandırılmadan tüm satır üzerinde aranır ve eşleşme bulunan kayıtlar sonuç listesine dâhil edilir. Bu yaklaşım, klasik alan bazlı aramalara kıyasla daha esnek ve kullanıcı dostu bir çözüm sunmaktadır.

Algoritmanın temelinde, CSV dosyasının pandas kütüphanesi ile DataFrame yapısına dönüştürülmesi yer almaktadır. Her bir satır, string türüne çevrilerek kullanıcının girdiği anahtar kelime ile karşılaştırılır. Büyük–küçük harf duyarlılığı kaldırılarak arama işlemi daha toleranslı hâle getirilmiştir. Böylece “Fiat”, “fiat” veya “FIAT” gibi farklı yazımlar aynı sonucu üretmektedir.

Örneğin;

Kullanıcı arama alanına “Fiat” ifadesini girdiğinde, markası Fiat olan tüm araçlar sonuç listesinde görüntülenmektedir.

Kullanıcı “34AB” şeklinde bir ifade girdiğinde, plakası bu karakterleri içeren araçlar da filtreleme sonucuna dâhil edilmektedir.

Benzer şekilde model adı veya araç ID’si gibi bilgiler de tek bir arama alanı üzerinden bulunabilmektedir.

Bu arama algoritması, sistemde tanımlı `_filter()` fonksiyonu aracılığıyla gerçekleştirilmiştir. Fonksiyon, ilgili CSV dosyasını okuyarak her satırı taramakta ve kullanıcının girdiği içeriğin herhangi bir sütunda bulunup bulunmadığını kontrol etmektedir. Eşleşme bulunan satırlar yeni bir liste hâlinde View katmanına aktarılmakta ve kullanıcıya sunulmaktadır.

Uygulanan bu yöntem sayesinde kullanıcılar, karmaşık filtreleme seçenekleriyle uğraşmadan, tek bir arama alanı üzerinden geniş kapsamlı sorgular yapabilmektedir. Bu durum hem kullanım kolaylığını artırmakta hem de sistemin erişilebilirliğini olumlu yönde etkilemektedir. Ayrıca algoritmanın modüler yapıda olması, ilerleyen aşamalarda gelişmiş filtreleme (fiyat aralığı, tarih uygunluğu vb.) özelliklerinin sisteme eklenmesine olanak tanımaktadır.

Carid	Marka	Model	Plaka	Payment	Owner
100	Toyota	Corolla	34ABC123	750	0
101	Honda	Civic	06XYZ456	820	0
102	Ford	Focus	35KLM789	680	0
103	Volkswagen	Golf	16QWE321	900	0
104	Renault	Megane	41RTY654	700	0
205	tofaş	A	25ER2525	900	0
222	Audi	A	34AA234	1200	0

Arabalar

Sepetim

Benim Arabalarım

Günlük Kiralama Özeti

Ayarlar

Arabalar

Carid	Marka	Model	Plaka	Payment	Owner
100	Toyota	Corolla	34ABC123	750	0
205	tofaş	A	25ER2525	900	0

Araç Filtrele:

To

Filtrele

Araba Ekle

Araç Kirala

Sepete Ekle

7. TESTLER VE SENARYOLAR

Geliştirilen araç kiralama uygulamasının doğruluğunu, güvenilirliğini ve kullanıcı beklentilerini karşılama düzeyini ölçmek amacıyla çeşitli test senaryoları uygulanmıştır. Bu testler, sistemin hem normal kullanım koşullarında hem de hatalı veya beklenmeyen kullanıcı girişleri karşısında nasıl davrandığını gözlemlemeyi amaçlamaktadır. Uygulanan testler, kullanıcı yönetimi, yetki+lendirme, veri tutarlılığı ve güvenlik başlıkları altında değerlendirilmiştir.

Test 1: “Şifremi Unuttum” Senaryosu

Bu test senaryosunda, sistemin e-posta tabanlı şifre kurtarma mekanizmasının doğru çalışıp çalışmadığı kontrol edilmiştir.

- **Giriş:** Sistemde kayıtlı olmayan bir e-posta adresi kullanıcı tarafından “Şifremi Unuttum” alanına girilmiştir.
- **Beklenen Davranış:** Sistem, girilen e-posta adresini **auth.csv** dosyasında aramalı ve herhangi bir eşleşme bulamazsa kullanıcıya “Kullanıcı bulunamadı” veya benzeri bir uyarı mesajı göstermelidir.
- **Gerçekleşen Sonuç:** Sistem, CSV dosyasını başarıyla taramış, girilen e-posta adresine ait bir kayıt bulamamış ve kullanıcıya hata mesajı göstermiştir. Herhangi bir e-posta gönderimi yapılmamıştır.

Bu test sonucunda, şifre kurtarma mekanizmasının hatalı veri girişlerine karşı doğru şekilde çalıştığı ve güvenlik açısından risk oluşturmadığı gözlemlenmiştir. Test başarıyla tamamlanmıştır.

Test 2: Kendi Aracını Silme Yetkisi

Bu test senaryosu, sistemde tanımlanan yetkilendirme kurallarının doğru uygulanıp uygulanmadığını ölçmek amacıyla gerçekleştirilmiştir.

- **Giriş:** Kullanıcı A, sistemde Kullanıcı B tarafından eklenmiş bir aracı silmeye çalışmıştır.

- **Beklenen Davranış:** Kullanıcılar yalnızca kendi ekledikleri araçlar üzerinde silme yetkisine sahip olmalı; başka bir kullanıcıya ait araçlar için silme işlemi engellenmelidir.
- **Gerçekleşen Sonuç:** Sistem, araç sahibi bilgisi ile aktif kullanıcı ID'sini karşılaştırmış ve eşleşme olmadığı için silme işlemini engellemiştir. Kullanıcı yalnızca kendi eklediği araçları silebilmiştir.

Bu test, sistemin yetkilendirme mekanizmasının doğru çalıştığını ve kullanıcılar arası veri güvenliğinin sağlandığını göstermektedir. Test başarıyla tamamlanmıştır.

Uygulanan testler sonucunda sistemin; hatalı girişlere karşı dayanıklı, yetkilendirme kurallarına uygun ve veri bütünlüğünü koruyan bir yapıya sahip olduğu görülmüştür. Test senaryoları, geliştirilen uygulamanın gerçek kullanım koşullarına hazır olduğunu göstermektedir.

8. SONUÇ VE GELECEK PLANLARI

Bu proje kapsamında, masaüstü ortamda çalışan, kullanıcı etkileşimli ve dosya tabanlı veri yönetimine sahip bir araç kiralama ve paylaşım platformu başarıyla geliştirilmiştir. Uygulama, Python programlama dili kullanılarak MVC (Model–View–Controller) mimarisine uygun biçimde tasarlanmış; kodun okunabilirliği, sürdürülebilirliği ve geliştirilebilirliği ön planda tutulmuştur. Geliştirilen sistem, kullanıcıların hem araç sahibi hem de kiracı rolünü üstlenebildiği hibrit yapısı ile klasik araç kiralama uygulamalarından ayrılmaktadır.

Projede merkezi bir yönetici (admin) rolünün bulunmaması, sistemin paylaşım temelli bir mantıkla çalışmasını sağlamıştır. Kullanıcıların özgürce araç ekleyebilmesi, araçlarını silebilmesi ve diğer kullanıcıların araçlarını kiralayabilmesi, uygulamayı P2P (Peer-to-Peer / Kişiden Kişiyeye) araç kiralama modeline yaklaştırmaktadır. Bu yaklaşım, günümüzde yaygınlaşan paylaşım ekonomisi anlayışıyla örtüşmekte ve sistemin modern yazılım çözümleri arasında konumlanmasına katkı sağlamaktadır.

CSV tabanlı dosya sistemi kullanılarak geliştirilen veri yönetim yapısı, küçük ve orta ölçekli uygulamalar için yeterli performans sunmuş; aynı zamanda sistemin kurulumu ve taşınabilirliği kolaylaştırılmıştır. Kullanıcı işlemleri, araç kayıtları, kiralama logları ve işlem geçmişleri ayrı dosyalarda tutulmuş, bu sayede veri bütünlüğü ve izlenebilirlik sağlanmıştır. Özellikle history ve logs mekanizmaları sayesinde sistemde yapılan tüm işlemler kayıt altına alınarak denetlenebilir bir yapı oluşturulmuştur.

Gelecek çalışmalar kapsamında, sistemin işlevselliğini ve analiz yeteneklerini artırmaya yönelik çeşitli geliştirmeler planlanmaktadır. Mevcut sürümde istatistiksel analizler (en çok kiralanan araç, kullanıcı bazlı kiralama sayıları, aylık veya yıllık ciro gibi metrikler) tutulmamaktadır. İlerleyen versiyonlarda, bu tür verilerin toplanması ve görselleştirilmesi amacıyla grafiksel istatistik modülleri eklenmesi hedeflenmektedir. Bu modüller sayesinde kullanıcılar veya yetkili roller, sistem performansını daha net bir şekilde analiz edebilecektir.

Bunun yanı sıra, mevcut CSV tabanlı veri yapısının ilerleyen aşamalarda SQL tabanlı bir veritabanı sistemine dönüştürülmesi planlanmaktadır. Bu dönüşüm, daha büyük veri hacimleriyle çalışılmasına olanak sağlayacak, eş zamanlı kullanıcı sayısını artıracak ve sistemin ölçeklenebilirliğini güçlendirecektir. Ayrıca web veya mobil platformlara taşınabilecek bir mimari altyapının oluşturulması da gelecekte değerlendirilebilecek geliştirmeler arasında yer almaktadır.

Sonu olarak bu proje, yazılım mhendislięi prensiplerine uygun, modler ve geniřletilebilir bir ara paylařım sistemi ortaya koymaktadır. Geliřtirilen uygulama, hem akademik hem de pratik aıdan bařarılı biralıřma olup, ileride daha kapsamlı ve profesyonel bir platforma dnřtrlebilecek saęlam bir temel sunmaktadır.